A Major Project report on

AI-Based Smart Traffic Violation Detection and Monitoring System

Major Project submitted to Anurag University in Partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Artificial Intelligence and Machine Learning.

Submitted by

ANWITH PULIJALA **22EG107C49**

Under the Guidance of

Mrs. S. Vijitha

Assistant professor



Department of Artificial Intelligence

School Of Engineering

ANURAG UNIVERSITY

2022-2026

ANURAG UNIVERSITY | SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE CERTIFICATE

This is to certify that the project report titled AI-Based Smart Traffic Violation Detection and Monitoring System is being submitted by Anwith Pulijala (22EG107C49), in IV B.Tech II semester ***Artificial Intelligence and Machine Learning*** is a record bonafide work carried out by them. The results embodied in this report have not been submitted to any other university for the award of any degree.

Student's Name	Student's Signature's
1. Anwith Pulijala	1.

Mrs. S. Vijitha	**Dr. A. Mallikarjun Reddy**	
Assistant Professor	**Associate Professor**	**External Examiner**
Guide	**Head of Department**	

ACKNOWLEDGEMENT

I express my sincere gratitude to Prof. Archana Mantri, Vice-Chancellor, Anurag University, for extending the university facilities that enabled the successful progress of my project and for her kind patronage.

I also wish to record my profound gratitude to Dr. V. Vijay Kumar, Dean – School of Engineering, for his constant motivation and encouragement throughout the project.

I sincerely thank Dr. A. Mallikarjuna Reddy, Associate Professor and Head of the Department of Artificial Intelligence, Anurag University, for providing me with all the necessary facilities and support in the successful completion of this project.

I would like to express my sincere gratitude to my project guide, Mrs. S. Vijitha, Assistant Professor, Department of Artificial Intelligence, Anurag University, for the valuable guidance, continuous support, and encouragement provided throughout this project. The guidance of my project guide has been instrumental in the successful completion of this project.

CONTENTS

ABSTRACT ..... **i**

LIST OF FIGURES ..... **ii**

LIST OF TABLES ..... **iii**

CHAPTER 1: INTRODUCTION **1**

CHAPTER 2: LITERATURE SURVEY **2**

2.1 EXISTING SYSTEMS 2

2.2 LIMITATIONS OF EXISTING SYSTEMS 3

2.2.1 Sensitivity to Lighting Conditions 3

2.2.2 High Computational Requirements 3

2.2.3 Motion Blur and Camera Quality Issues 3

2.2.4 Limited Multi-Violation Detection 3

2.2.5 Poor Performance Under Occlusion 3

2.2.6 Lack of Real-Time Integrated Monitoring 3

2.3 GAPS IDENTIFIED 4

2.3.1 Lack of Unified Detection Systems 4

2.3.2 Limited Use of Advanced Models 4

2.3.3 Insufficient Real-World Robustness 4

2.3.4 Absence of Reliable Tracking Mechanisms 4

2.3.5 Limited Dataset Size and Diversity 4

2.3.6 Incomplete Violation Reporting 4

2.4 PROBLEM STATEMENT 5

2.5 OBJECTIVES 6

CHAPTER 3: PROPOSED SYSTEM **7**

3.1 SYSTEM ARCHITECTURE 7

3.1.1 Input Layer 7

3.1.2 Detection Model Layer (YOLOv8) 8

3.1.3 Tracking Layer 8

3.1.4 Recognition Layer (ANPR) 8

3.1.5 Server Layer (FastAPI Backend) 8

3.1.6 Database Layer 9

3.1.7 Dashboard Layer 9

3.2 DATA FLOW 9
3.2.1 Video/Image Input 9
3.2.2 Frame Extraction and Preprocessing 10
3.2.3 Object Detection and Classification 11
3.2.4 Rider Tracking 11
3.2.5 Number Plate Recognition 11
3.2.6 Violation Validation and Data Storage 11
3.2.7 Dashboard Display and Monitoring 11
3.3 SEQUENCE DIAGRAM 12
3.3.1 Components Involved 12
3.4 REQUIREMENTS AND SPECIFICATIONS 14
3.4.1 Client Requirements 14
3.4.2 Software Requirements 15
3.4.3 Hardware Requirements 16
CHAPTER 4: METHODOLOGY 17
4.1 DATA COLLECTION 17
4.2 DATA PREPROCESSING 18
4.3 MODEL DESCRIPTION 18
4.3.1 Overview of YOLOv8l 19
4.3.2 Architecture of YOLOv8l 19
4.4 TRAINING PROCEDURE 20
4.5 MODEL EVALUATION 21
CHAPTER 5: IMPLEMENTATION & TESTING 23
5.1 IMPLEMENTATION OVERVIEW 23
5.2 FRONTEND IMPLEMENTATION 24
5.3 BACKEND IMPLEMENTATION 24
5.4 MODEL INTEGRATION 25
5.5 DATABASE IMPLEMENTATION 26
5.6 TESTING 27
5.6.1 Objectives of Testing 28
5.6.2 Functional Testing 28
5.6.3 Performance Testing 29
5.6.4 Integration Testing 30

5.6.5 User Testing 31
5.6.6 Testing Summary 31
CHAPTER 6: RESULTS AND DISCUSSION 32
6.1 MODEL PERFORMANCE 32
6.2 COMPARATIVE ANALYSIS 34
6.3 WEB APPLICATION RESULTS & OUTPUTS 35
6.4 DISCUSSION 41
CHAPTER 7: CONCLUSION 43
CHAPTER 8: FUTURE WORK 44
8.1 FUTURE ENHANCEMENTS 44
REFERENCES 46
ANNEXURE iv

ABSTRACT

Road accidents involving two-wheeler riders continue to be a major safety concern, primarily due to the non-use of helmets and weak enforcement of traffic regulations. Traditional traffic monitoring methods rely heavily on manual supervision, which is time-consuming, inefficient, and difficult to scale in high-traffic environments. To address these challenges, this project proposes an AI-Based Smart Traffic Violation Detection and Monitoring System that automatically detects traffic violations using computer vision and deep learning techniques. The proposed system utilizes the YOLOv8 deep learning model to detect key entities such as riders, helmets, no-helmet cases, and number plates from real-time video streams. To enhance detection reliability, a rider tracking mechanism is incorporated to consistently track the same rider across multiple frames. When a helmet violation is detected, an Automatic Number Plate Recognition module extracts the vehicle number for identification and further action. The backend of the system is developed using FastAPI, which handles model inference, violation processing, and communication with a centralized monitoring dashboard. All detected violations, along with details such as vehicle number, timestamp, and location, are stored and displayed for traffic authorities to monitor and analyze effectively. The system aims to reduce dependency on manual surveillance, improve the efficiency of traffic rule enforcement, and enhance overall road safety. Furthermore, the solution is designed to be scalable and can be integrated into smart traffic management systems and smart city infrastructures.

Keywords: Traffic Violation Detection, YOLOv8, Deep Learning, Computer Vision, Helmet Detection, Rider Tracking, Automatic Number Plate Recognition (ANPR), Object Detection, Smart Traffic Monitoring, Real-Time Surveillance, FastAPI, Smart City

LIST OF FIGURES

Figure No	Figure Name	Page No
Figure 3.1	System Architecture	7
Figure 3.2	Data Flow / Communication Pipeline	10
Figure 3.3	Sequence Diagram	12
Figure 6.1	Confusion Matrix	33
Figure 6.2	Training Performance Graph of YOLOv8l Model	33
Figure 6.3	Validation Batch Prediction Results	35
Figure 6.4	Helmet Detection Output on Multiple Riders	36
Figure 6.5	Rider, Helmet, and Number Plate Detection Output	37
Figure 6.6 (a)	Home Page of Detection Website	38
Figure 6.6 (b)	Image Upload Interface for Processing	38
Figure 6.6 (c)	Final detection output generated by the system	39
Figure 6.7 (a)	Supabase Detection Records Table	40
Figure 6.7 (b)	Supabase Storage Bucket for Processed Outputs	41

LIST OF TABLES

Table No	Table Name	Page No
Table 2.1	Summary of Existing Systems	2
Table 3.1	Software Requirements	15
Table 3.2	Hardware Requirements	16
Table 4.1	Data Collection	17
Table 4.2	Model Training Parameters	21
Table 4.3	Evaluation Metrics	22
Table 5.2	Database Fields	27
Table 5.3	Functional Testing	29
Table 5.4	Performance Testing	30
Table 6.1	Model Performance	32
Table 6.2	Comparative Analysis of Detection Models	34

CHAPTER 1: INTRODUCTION

Road safety has become a major concern in today's rapidly growing urban environments, especially with the increasing number of two-wheelers on the road. Violations such as riding without helmets, overloading, and lack of proper vehicle identification significantly contribute to road accidents and fatalities. Among these, helmet non-compliance is one of the leading causes of severe injuries in two-wheeler accidents. Ensuring strict enforcement of traffic rules is therefore essential to improve public safety.

Traditionally, traffic monitoring is carried out using traffic police personnel and CCTV surveillance systems. While these methods help in observing traffic conditions, they rely heavily on manual supervision to detect violations. This approach is time-consuming, prone to human error, and not scalable for monitoring large and densely populated urban areas. As traffic volume continues to grow, there is a clear need for an automated and intelligent system that can monitor violations efficiently and in real time.

Recent advancements in Artificial Intelligence (AI) and Computer Vision have enabled the development of smart systems capable of analyzing images and video streams automatically. Deep learning-based object detection models, particularly the YOLO (You Only Look Once) family, have gained significant attention due to their ability to perform real-time detection with high accuracy. The latest version, YOLOv8, offers improved performance, speed, and flexibility, making it suitable for real-world traffic monitoring applications. In this project, we propose an AI-Based Smart Traffic Violation Detection and Monitoring System that leverages YOLOv8 to detect multiple classes including rider, helmet, no-helmet, and number plate from real-time video streams. To enhance system reliability, a rider tracking mechanism is incorporated to maintain consistency across consecutive frames. Additionally, an Automatic Number Plate Recognition (ANPR) module is used to extract vehicle numbers when violations are detected, enabling proper identification of offenders.

The system uses a FastAPI-based backend to handle model inference, data processing, and dashboard communication. Detected violations, along with details such as timestamp and location, are stored for analysis by traffic authorities. The objective is to develop an efficient, scalable, and automated traffic violation detection system that reduces manual effort and improves road safety.

CHAPTER 2: LITERATURE SURVEY

2.1 EXISTING SYSTEMS

A comprehensive literature review was conducted to analyze existing research on Traffic violation detection systems.

Table 2.1: Summary of Existing Systems

S.No	Author & Year	Approach / Model	Key Contributions	Limitations
1	Deshpande et al., 2025	YOLOv8-based Helmet Detection	Achieved high accuracy in detecting helmet and no-helmet cases in real-time traffic scenarios; effective in handling multiple objects	Performance affected under varying lighting conditions; no number plate recognition
2	Hegde et al., 2025	Helmet Detection + License Plate Recognition	Integrated violation detection with vehicle identification; suitable for real-world monitoring	Requires high computational resources; struggles with motion blur
3	Adak et al., 2022	YOLO + CNN for Number Plate Recognition	Combined detection and character extraction; reliable results in controlled environments	Poor performance under occlusion and complex backgrounds
4	Soltanikazemi et al., 2023	YOLOv5-based Helmet Detection	Balanced detection speed and accuracy; suitable for real-time applications	Does not support multi-violation detection or ANPR integration
5	Wu et al., 2025	Improved YOLOv7 Helmet Detection	Enhanced detection accuracy using architectural improvements	Increased model complexity and higher computational cost

2.2 LIMITATIONS OF EXISTING SYSTEMS

Although several research studies have shown promising results in traffic violation detection using deep learning and computer vision, many existing systems still face practical limitations when deployed in real-world environments.

2.2.1 Sensitivity to Lighting Conditions

Many detection models perform well in controlled environments but show reduced accuracy under poor lighting, shadows, glare, rain, or night-time conditions. These variations affect the visibility of helmets, riders, and number plates.

2.2.2 High Computational Requirements

Some advanced deep learning models require powerful GPUs and high-end hardware for training and real-time inference. This increases deployment cost and makes implementation difficult in low-resource environments.

2.2.3 Motion Blur and Camera Quality Issues

In real-time traffic monitoring, moving vehicles often cause motion blur, especially in high-speed scenarios. Low-resolution cameras and unstable video feeds can further reduce detection and recognition accuracy.

2.2.4 Limited Multi-Violation Detection

Many existing systems focus only on a single task such as helmet detection or number plate recognition. They do not provide a complete solution capable of detecting multiple violations simultaneously.

2.2.5 Poor Performance Under Occlusion

Traffic scenes often contain overlapping vehicles, pedestrians, and obstacles. In such cases, helmets or number plates may be partially hidden, leading to missed detections or incorrect predictions.

2.2.6 Lack of Real-Time Integrated Monitoring

Several proposed systems focus mainly on model development and testing but do not include dashboards, alert systems, or centralized monitoring tools for traffic authorities.

2.3 GAPS IDENTIFIED

Based on the analysis of existing systems and research studies, several important gaps have been identified in current traffic violation detection approaches. These gaps highlight the need for a more efficient, accurate, and practical solution.

2.3.1 Lack of Unified Detection Systems

Most existing systems focus on a single task such as helmet detection or number plate recognition. Very few systems combine rider detection, helmet compliance checking, tracking, and number plate extraction within one integrated framework.

2.3.2 Limited Use of Advanced Models

Some earlier studies use older object detection models that provide lower accuracy or slower inference speed compared to recent versions such as YOLOv8. There is a need to utilize modern models for better real-time performance.

2.3.3 Insufficient Real-World Robustness

Many systems are tested in controlled environments and do not perform consistently under real-world challenges such as low lighting, weather changes, motion blur, heavy traffic, and camera angle variations.

2.3.4 Absence of Reliable Tracking Mechanisms

Several existing approaches detect violations frame by frame but do not track the same rider or vehicle across multiple frames. This may lead to duplicate detections or missed violations.

2.3.5 Limited Dataset Size and Diversity

A number of studies rely on small or less diverse datasets. This reduces the model's ability to generalize effectively across different traffic conditions, road environments, and vehicle types.

2.3.6 Incomplete Violation Reporting

Many systems stop at detection and do not provide complete reporting features such as timestamp, location, vehicle number, or centralized dashboard monitoring for authorities.

2.4 PROBLEM STATEMENT

Traffic violations involving two-wheelers have become a major concern in many urban and semi-urban areas. Common violations such as riding without helmets, carrying multiple riders, and using vehicles without proper identification significantly increase the risk of accidents and serious injuries. Among these, helmet non-compliance remains one of the leading causes of fatalities in two-wheeler road accidents.

At present, traffic monitoring is largely dependent on traffic police personnel and CCTV surveillance systems. Although these methods help in observing road activities, they require continuous manual supervision to identify violations. This process is time-consuming, labour-intensive, and often ineffective in areas with heavy traffic movement. In many cases, violations go unnoticed due to limited manpower and the inability to monitor multiple locations simultaneously.

Existing automated traffic monitoring systems have attempted to solve this problem using image processing and deep learning techniques. However, many of these systems focus only on a single task such as helmet detection or number plate recognition. Some models also struggle under real-world conditions such as low lighting, motion blur, camera angle variations, occlusion, and dense traffic scenes. In addition, several systems require high computational resources, making large-scale deployment difficult.

Therefore, there is a need for an intelligent, accurate, and scalable traffic monitoring solution that can automatically detect multiple violations in real time. The system should be capable of identifying riders, checking helmet usage, tracking violations across frames, and extracting number plate details for proper enforcement. Such a solution can reduce dependence on manual monitoring, improve traffic rule enforcement, and enhance overall road safety.

2.5 OBJECTIVES

The main objectives of the proposed AI-Based Smart Traffic Violation Detection and Monitoring System are as follows:

1. To develop an intelligent traffic monitoring system capable of automatically detecting traffic violations using Artificial Intelligence and Computer Vision techniques.
2. To implement the YOLOv8 deep learning model for real-time object detection with high accuracy and speed.
3. To detect multiple classes such as rider, helmet, no-helmet, and number plate from images and video streams.
4. To incorporate a rider tracking mechanism for maintaining consistent detection across multiple video frames and reducing duplicate violations.
5. To integrate Automatic Number Plate Recognition (ANPR) for extracting vehicle registration numbers when violations are detected.
6. To build a backend system using FastAPI for model inference, data processing, and communication with the monitoring dashboard.
7. To store and display detected violations along with relevant details such as vehicle number, timestamp, and location for traffic authorities.
8. To reduce dependence on manual traffic surveillance and improve the efficiency of traffic rule enforcement.
9. To enhance overall road safety by promoting helmet compliance and timely identification of violations.
10. To design a scalable solution that can be integrated into smart traffic management systems and future smart city infrastructures.

CHAPTER 3: PROPOSED SYSTEM

3.1 SYSTEM ARCHITECTURE

The AI-Based Smart Traffic Violation Detection and Monitoring System is designed as an intelligent and modular framework for detecting traffic violations automatically in realtime. The system combines deep learning, computer vision, rider tracking, number plate recognition, and backend technologies to improve the efficiency of traffic monitoring. Each layer of the architecture performs a specific task and works together to provide accurate detection, reliable monitoring, and effective reporting of violations.

The architecture is divided into seven major layers: Input Layer, Detection Model Layer, Tracking Layer, Recognition Layer, Server Layer, Database Layer, and Dashboard Layer. The coordinated functioning of these layers ensures smooth data processing from video capture to final violation reporting.

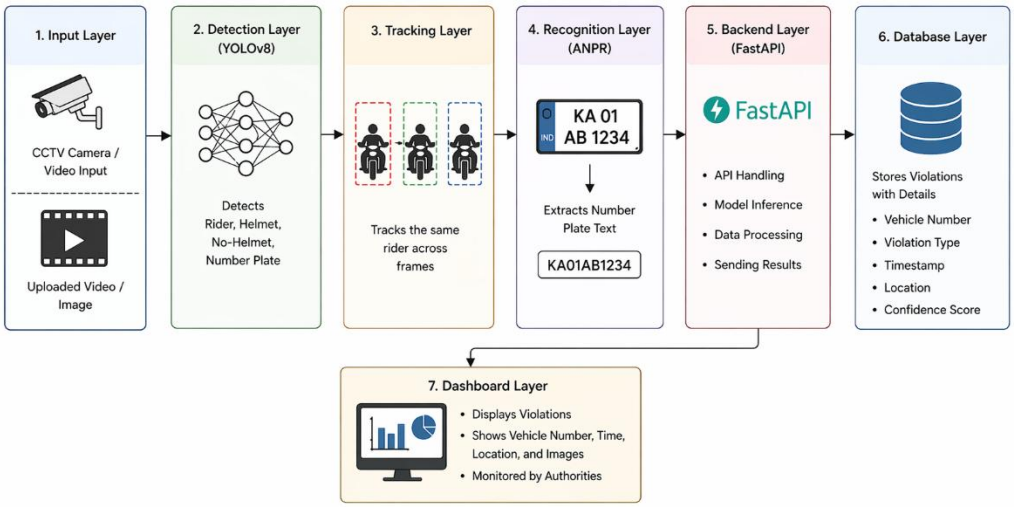


Figure 3.1: System Architecture

3.1.1 Input Layer

The Input Layer is the first stage of the system where traffic data is collected. The input can be obtained from live CCTV cameras, road surveillance cameras, traffic junction cameras, or pre-recorded video files. The system can also process image inputs when required for testing and validation purposes. The incoming video stream is divided into individual frames so that each frame can be analyzed efficiently. This layer is

responsible for providing clear and continuous visual data to the detection model for further processing.

3.1.2 Detection Model Layer (YOLOv8)

The Detection Model Layer is the core component of the system and is powered by the YOLOv8 deep learning model. YOLOv8 is selected because of its high detection accuracy, faster inference speed, and suitability for real-time object detection tasks.

The trained model detects multiple traffic-related classes such as rider, helmet, no-helmet, and number plate from each input frame. It generates bounding boxes around detected objects along with confidence scores. This layer plays a major role in identifying traffic violations accurately under different road conditions.

3.1.3 Tracking Layer

The Tracking Layer is responsible for maintaining the identity of detected riders or vehicles across consecutive frames. In real-time video streams, the same rider appears in multiple frames, and repeated detection may lead to duplicate violation records.

To overcome this issue, a rider tracking mechanism is used to follow the movement of the same rider continuously. This improves consistency in detection and ensures that a single violation is recorded only once for the same vehicle.

3.1.4 Recognition Layer (ANPR)

The Recognition Layer performs Automatic Number Plate Recognition (ANPR). Whenever a violation such as no-helmet usage is detected, the system extracts the number plate region from the frame.

After extracting the number plate area, optical character recognition techniques are used to convert the plate image into readable text. This helps in identifying the violating vehicle and supports further enforcement actions by traffic authorities.

3.1.5 Server Layer (FastAPI Backend)

The Server Layer is developed using FastAPI and acts as the communication bridge between the frontend, detection modules, and database. It handles API requests, frame processing, model inference, and transfer of detection results. FastAPI is chosen because it provides fast response time, efficient asynchronous processing, and smooth

integration with machine learning models. This layer ensures that the complete system works efficiently in real-time scenarios.

3.1.6 Database Layer

The Database Layer is used to store all detected violation records and related information. Important details such as vehicle number, timestamp, detected class, confidence score, and location are saved for future reference. This centralized storage system helps in maintaining traffic records, generating reports, and analyzing repeated violations. It also supports long-term monitoring and decision-making by authorities.

3.1.7 Dashboard Layer

The Dashboard Layer provides a user-friendly monitoring interface for traffic authorities. All detected violations are displayed in a structured format with relevant information such as vehicle number, date, time, and violation type. The dashboard enables officials to monitor traffic conditions, verify violations, and take necessary action quickly. It also helps in visual analysis and efficient management of traffic enforcement activities.

The proposed layered architecture ensures scalability, faster processing, accurate violation detection, and easy integration with future smart traffic management systems and smart city infrastructure.

3.2 DATA FLOW

The data flow of the proposed AI-Based Smart Traffic Violation Detection and Monitoring System describes how information moves through different modules of the system, starting from traffic input capture to final violation reporting. It provides a clear understanding of how video or image data is processed, analyzed, stored, and displayed for traffic authorities. The complete workflow is designed to ensure accurate real-time monitoring and efficient traffic rule enforcement.

3.2.1 Video/Image Input

The process begins when traffic data is provided to the system through live CCTV cameras, surveillance cameras, uploaded video files, or image inputs. These input sources capture road activities such as vehicle movement, rider behavior, and traffic violations. The quality of the input data plays an important role in achieving accurate detection results.

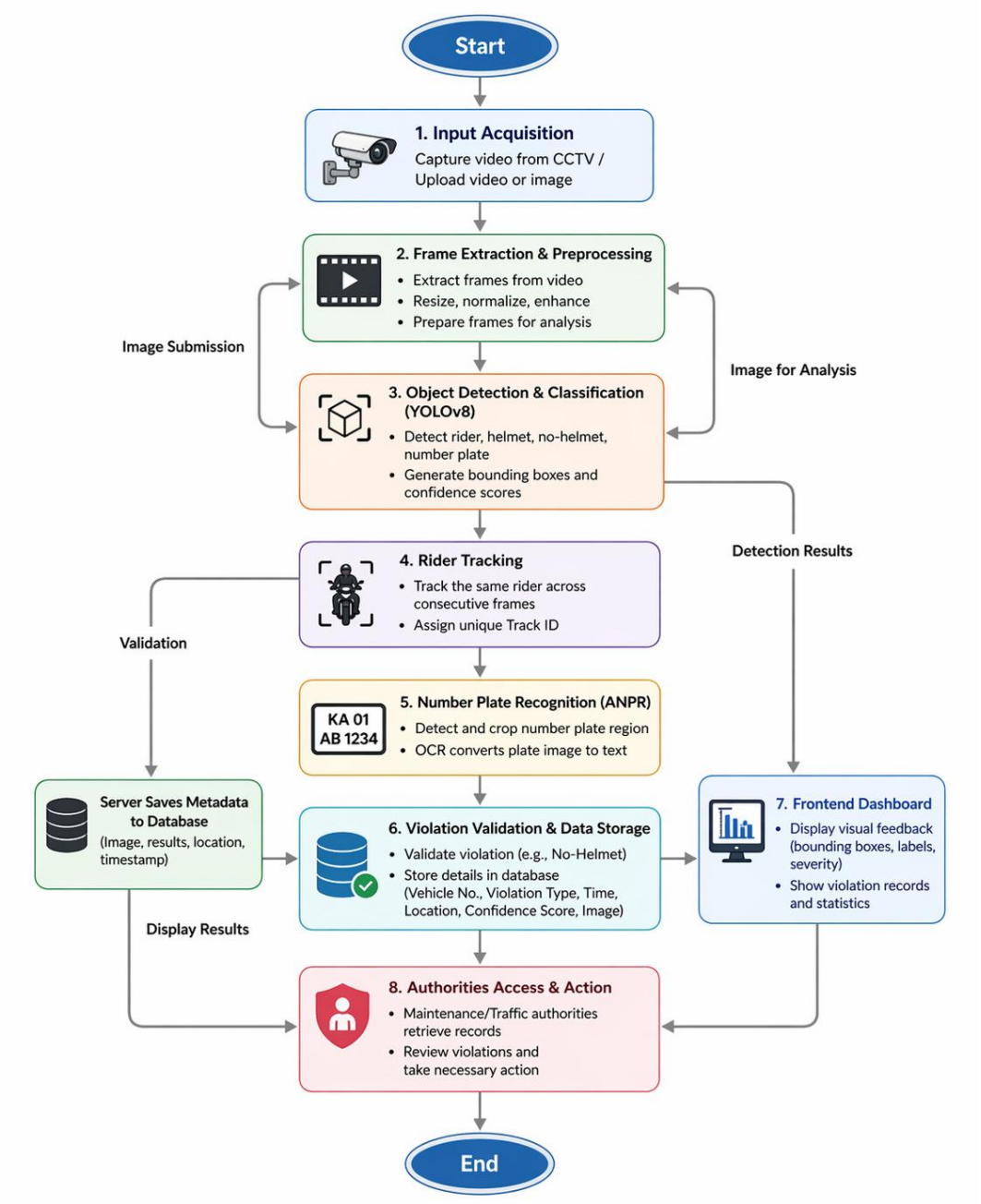


Figure 3.2: Data Flow / Communication Pipeline

3.2.2 Frame Extraction and Preprocessing

If the input is in video format, the video stream is divided into individual frames for analysis. Each frame is then preprocessed to improve detection quality. Common preprocessing steps include resizing frames to suitable dimensions, adjusting brightness if required, and converting data into a format accepted by the YOLOv8 model. This stage helps improve model performance and processing speed.

3.2.3 Object Detection and Classification

After preprocessing, each frame is passed to the YOLOv8 detection model. The model analyzes the frame and identifies multiple classes such as rider, helmet, no-helmet, and number plate. Bounding boxes are generated around detected objects along with confidence scores. This stage is the core of the system, where traffic violations are initially recognized.

3.2.4 Rider Tracking

Once objects are detected, the rider tracking module follows the same rider or vehicle across consecutive frames. Since a video contains many frames, the same rider may appear repeatedly. Tracking helps maintain a unique identity for each rider and prevents duplicate violation entries. It also improves consistency in monitoring moving vehicles.

3.2.5 Number Plate Recognition

When a violation such as no-helmet usage is confirmed, the detected number plate region is extracted from the frame. The Automatic Number Plate Recognition (ANPR) module then processes the cropped plate image and converts the characters into readable text. This enables identification of the violating vehicle for further action.

3.2.6 Violation Validation and Data Storage

After successful detection and recognition, the system validates the violation and stores all relevant details in the database. The stored information may include vehicle number, type of violation, timestamp, confidence score, and location details. This centralized data storage helps maintain proper records for future reference and analysis.

3.2.7 Dashboard Display and Monitoring

Finally, all recorded violations are displayed on the monitoring dashboard for traffic authorities. The dashboard provides an organized view of detected violations and allows officials to review records, search entries, and generate reports. This helps in taking timely enforcement actions and improving traffic management efficiency.

The data flow of the proposed system ensures smooth communication between all modules and enables accurate, automated, and real-time traffic violation monitoring.

3.3 SEQUENCE DIAGRAM

The sequence diagram of the proposed AI-Based Smart Traffic Violation Detection and Monitoring System illustrates the interaction between different system components in a time-based order. It explains how requests are processed from input acquisition to final violation reporting. The sequence of operations ensures smooth communication between modules and supports accurate real-time traffic monitoring.

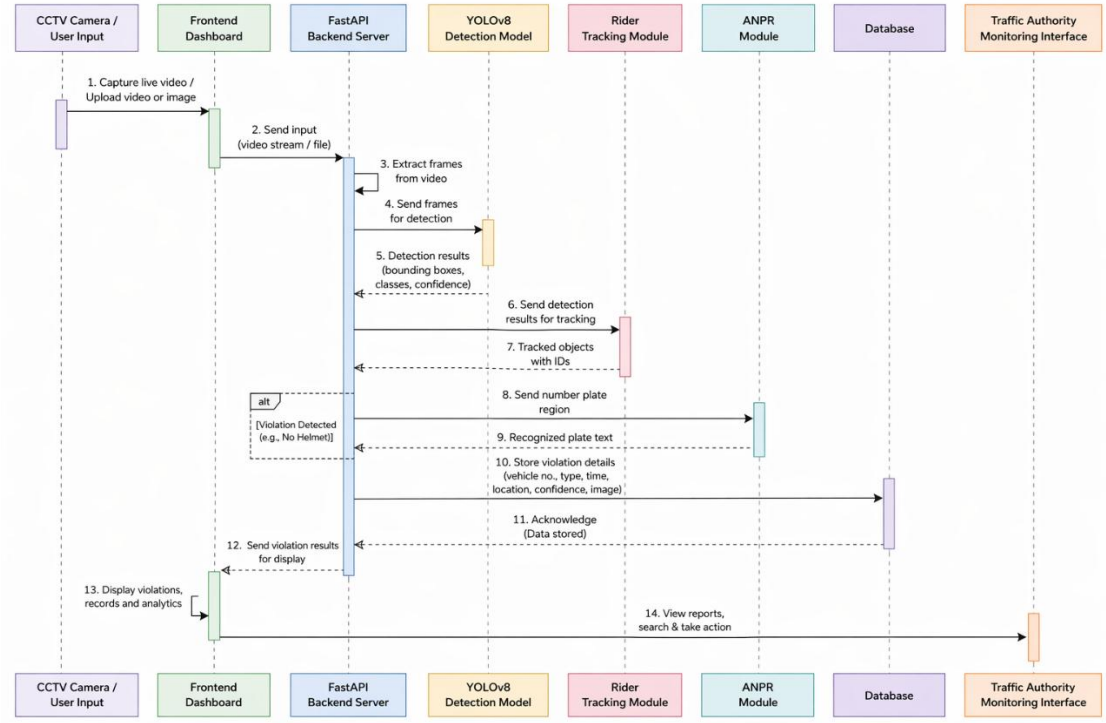


Figure 3.3: Sequence Diagram

3.3.1 Components Involved

The major components involved in the sequence flow of the proposed system are described below:

1. **CCTV Camera / User Input Module**

   This component is responsible for providing input data to the system. It captures live traffic footage through surveillance cameras or accepts uploaded video and image files for analysis.

2. **Frontend Dashboard**

   The frontend dashboard acts as the user interface of the system. It displays detected violations, monitoring statistics, vehicle details, and allows authorities to review records efficiently.

3. **FastAPI Backend Server**

   The backend server manages communication between all modules of the system. It handles API requests, frame processing, model inference, and transfers results to the dashboard and database.

4. **YOLOv8 Detection Model**

   This is the core detection engine of the system. It analyzes frames and identifies classes such as rider, helmet, no-helmet, and number plate with high speed and accuracy.

5. **Rider Tracking Module**

   The tracking module follows the same rider or vehicle across multiple consecutive frames. It helps maintain a unique identity and prevents duplicate violation entries.

6. **Number Plate Recognition (ANPR) Module**

   This module extracts the number plate region from detected vehicles and converts the plate image into readable text using OCR techniques. It helps identify violating vehicles accurately.

7. **Database (Supabase)**

   The database stores all violation-related information for future reference. It saves details such as vehicle number, violation type, timestamp, confidence score, and location.

8. **Traffic Authority Monitoring Interface**

   This component is used by traffic officials to monitor detected violations and review stored records. It supports decision-making, report generation, and enforcement actions.

3.4 REQUIREMENTS AND SPECIFICATIONS

The successful implementation of the proposed AI-Based Smart Traffic Violation Detection and Monitoring System depends on clearly defined functional and technical requirements. These requirements help ensure that the system operates efficiently, meets user expectations, and supports real-time traffic monitoring. The following section describes the client requirements, software requirements, and hardware requirements of the proposed system.

3.4.1 Client Requirements

Client requirements describe what the end users and stakeholders expect from the system. In this project, the major users include traffic authorities, administrators, and monitoring personnel who access the dashboard and review detected violations. These requirements ensure that the final system is practical, user-friendly, and suitable for real-world deployment.

1. **Accessibility:** The application should be accessible through modern web browsers and authorized devices without requiring complex installation procedures.
2. **Ease of Use:** The user interface should be simple and easy to understand, allowing users to monitor violations, search records, and review reports efficiently.
3. **Real-Time Monitoring:** The system should provide real-time updates whenever traffic violations are detected from live CCTV feeds or uploaded videos.
4. **Accurate Violation Reporting:** Users should receive clear details such as vehicle number, violation type, timestamp, and location for every detected case.
5. **Secure Data Handling:** Violation records, images, and sensitive information must be stored securely with proper authentication and access control.
6. **Authority Dashboard:** Traffic officials should have access to a centralized dashboard where they can monitor violations, filter records, and analyze statistics.
7. **Scalability and Availability:** The system should support multiple cameras or users simultaneously and remain available continuously for monitoring operations.

3.4.2 Software Requirements

The proposed system uses various software tools and frameworks for model training, backend development, database handling, and user interface support.

Table 3.1: Software Requirements

S.No	Category	Software / Tool	Purpose / Description
1	**Operating System**	Windows / Linux / macOS	Platform for development and deployment
2	**Programming Language**	Python	Core language used for model training and backend development
3	**Deep Learning Framework**	Ultralytics YOLOv8	Used for object detection and real-time inference
4	**Annotation Tool**	Roboflow	Used for dataset labeling, preprocessing, and augmentation
5	**Backend Framework**	FastAPI	Handles APIs, model inference, and communication
6	**Frontend Framework**	HTML, CSS, JavaScript / ReactJS	Used to develop dashboard and user interface
7	**Cloud Database**	Supabase (PostgreSQL)	Stores violation records, images, and metadata in cloud environment
8	**Computer Vision Library**	OpenCV	Used for video processing and frame handling
9	**OCR Tool**	EasyOCR / Tesseract OCR	Used for number plate text extraction
10	**Development Environment**	VS Code / PyCharm / Google Colab	Used for coding, training, and testing
11	**Version Control**	GitHub	Used for source code management and collaboration

3.4.3 Hardware Requirements

The hardware resources are important for training the YOLOv8 model and executing real-time traffic monitoring tasks efficiently.

Table 3.2: Hardware Requirements

S.No	Category	Minimum Requirement	Recommended Requirement
1	**Processor**	Intel Core i5 / AMD Equivalent	Intel Core i7 / AMD Ryzen 7 or above
2	**Memory (RAM)**	8 GB	16 GB or above
3	**Storage**	256 GB HDD / SSD	512 GB SSD or above
4	**Graphics Unit**	Integrated Graphics	NVIDIA GPU with CUDA support
5	**Display**	1366 × 768 Resolution	Full HD (1920 × 1080)
6	**Internet**	Stable Broadband Connection	High-Speed Internet Connection
7	**Camera Input**	Webcam / CCTV Camera	HD CCTV Camera with continuous feed

CHAPTER 4: METHODOLOGY

4.1 DATA COLLECTION

The dataset used in this project was prepared by collecting images from publicly available traffic monitoring sources and custom road surveillance data. It consists of approximately 14,000 images representing real-world two-wheeler traffic scenarios. The collected dataset includes different riding conditions such as single riders, multiple riders, riders wearing helmets, riders without helmets, and visible vehicle number plates. Since the effectiveness of any deep learning model depends greatly on the quality and diversity of the training data, special attention was given to collecting images from multiple traffic environments. The data collection process focused on capturing realistic road conditions under different situations such as urban roads, highways, traffic junctions, and crowded streets. Images with variations in lighting, camera angles, motion blur, etc were also included to improve model generalization. This diversedataset helps the model learn robust features and perform accurately in real-time traffic violation detection.

Table 4.1: Data Collection

S.No	Category	Details	Remarks
1	Total Images	14,392	Complete dataset collected for training and evaluation
2	Training Set (70%)	10,074 Images	Used for model learning
3	Validation Set (20%)	2,878 Images	Used for tuning performance
4	Testing Set (10%)	1,440 Images	Used for final testing
5	Number of Classes	4	Helmet, No-Helmet, Number Plate, Rider
6	Data Source	Public + Custom Images	Real traffic monitoring data
7	Image Conditions	Day/Night, Blur, Shadows, Angles	Improves robustness
8	Dataset Type	Annotated Object Detection Dataset	Prepared using Roboflow

4.2 DATA PREPROCESSING

Before feeding the dataset into the YOLOv8 model, several preprocessing steps were applied to improve image quality and training efficiency:

1. **Resizing:** All images were resized to a fixed input resolution suitable for YOLOv8 training to maintain uniformity and improve processing speed.
2. **Normalization:** Pixel values were normalized to an appropriate range, helping the model learn features more effectively and ensuring stable training.
3. **Data Augmentation:** Techniques such as horizontal flipping, rotation, brightness adjustment, scaling, and noise variation were applied to improve model generalization under real-world conditions.
4. **Annotation Verification:** Bounding box labels for classes such as helmet, no-helmet, rider, and number plate were checked and corrected to ensure annotation accuracy.
5. **Annotation Conversion:** Bounding box coordinates were converted into YOLO format (class, x_center, y_center, width, height) for model compatibility.
6. **Dataset Splitting:** The final dataset was divided into training, validation, and testing sets in the ratio of 70%, 20%, and 10% respectively.

4.3 MODEL DESCRIPTION

The proposed AI-Based Smart Traffic Violation Detection and Monitoring System uses the YOLOv8l object detection model for detecting traffic violations in real time. YOLOv8l is a large variant of the YOLOv8 family and is designed to provide higher detection accuracy through deeper feature extraction and stronger learning capability. It is widely used in computer vision applications where both speed and precision are important.

In this project, YOLOv8l was trained to detect four important classes: helmet, no-helmet, rider, and number plate. The model processes images and video frames to generate bounding boxes, class labels, and confidence scores. These outputs are further used for rider tracking, number plate recognition, and violation monitoring.

4.3.1 Overview of YOLOv8l

The YOLOv8 family consists of multiple model variants such as YOLOv8n (nano), YOLOv8s (small), YOLOv8m (medium), YOLOv8l (large), and YOLOv8x (extra-large). Each variant is designed to balance speed, model size, and detection accuracy based on available computational resources.

• **YOLOv8n** is the most lightweight and fastest version, suitable for edge devices and low-resource systems.

• **YOLOv8s** provides better accuracy than the nano model while maintaining high inference speed.

• **YOLOv8m** offers a balanced trade-off between speed and precision for medium-scale applications.

• **YOLOv8l** provides higher detection accuracy with stronger feature extraction, making it suitable for complex real-time applications.

• **YOLOv8x** delivers the highest accuracy among all variants but requires more GPU memory and computational power.

For this project, YOLOv8l was selected because it offers an effective balance between accuracy and processing speed. It performs well in detecting small and partially visible objects such as helmets and number plates in crowded traffic scenes.

YOLOv8l also supports transfer learning, allowing pre-trained weights to be fine-tuned on custom datasets. This helped in training the model efficiently on the traffic violation dataset used in this project.

4.3.2 Architecture of YOLOv8l

The architecture of YOLOv8l consists of three major components: Backbone, Neck, and Head. These components work together to perform accurate and fast object detection.

1. **Backbone:**

   The backbone extracts meaningful visual features from the input image such as edges, shapes, textures, and object structures. Since YOLOv8l has a deeper backbone, it can capture richer features for improved detection performance.

2. **Neck:**

   The neck combines feature maps from multiple levels and enhances multi-scale feature representation. This is useful for detecting small objects like number plates as well as larger objects like riders.

3. **Head:**

   The head is the final prediction layer that generates bounding boxes, class labels, and confidence scores for detected objects.

During inference, the input image passes through these three stages and the final predictions are produced in real time. The efficient and powerful architecture of YOLOv8l makes it highly suitable for traffic surveillance systems where accuracy and reliability are essential.

4.4 TRAINING PROCEDURE

Training of the proposed YOLOv8l model was carried out in two experimental phases using different dataset sizes and training environments. During the training process, model checkpoints were saved automatically, and validation performance was monitored continuously to obtain the best model weights.

In the initial phase, the YOLOv8l model was trained on a dataset containing 1,810 images using Google Colab with GPU acceleration. This phase helped in understanding baseline model performance, validating the dataset quality, and identifying areas for further improvement.

In the second phase, the dataset was significantly expanded to 14,392 images with better diversity and improved annotations. The model was retrained locally using a dedicated GPU system, which provided greater control over training configuration and faster experimentation. This second training phase resulted in improved detection accuracy, better robustness, and stronger generalization performance. Therefore, the best model weights obtained from the second phase were selected for final deployment.

Table 4.2: Model Training Parameters

Parameter	Description	Value Used
Model	YOLOv8l	Large variant used for high accuracy detection
Image Size	Input image resolution	640 × 640 px
Batch Size	Number of images processed per iteration	8
Epochs	Number of complete passes through dataset	120
Optimizer	Algorithm for updating weights	Adam / SGD
Learning Rate	Speed of weight updates	0.001
Workers	Parallel data loading threads	8
Loss Function	Calculates prediction error	Box Loss + Class Loss + DFL
Augmentation	Data transformations	Flip, rotate, brightness
Patience	Early stopping parameter	20
Device	GPU used for final training	NVIDIA GeForce RTX 3050 6GB Laptop GPU

During training, the model learned to detect four custom classes: helmet, no-helmet, rider, and number plate. After successful completion of training, the best-performing weights were selected for model evaluation and real-time deployment.

4.5 MODEL EVALUATION

After training, the performance of the YOLOv8l model was evaluated using standard object detection metrics. The model was tested on unseen images from the testing dataset to measure its detection capability and generalization performance. The major evaluation metrics used in this project are listed below:

- **Precision (P):** Measures the accuracy of positive detections.
- **Recall (R):** Measures the ability to detect all relevant objects present in the dataset.
- **F1-Score:** Harmonic mean of precision and recall.

- **mAP (Mean Average Precision):** Measures detection accuracy across IoU thresholds.
- **Confusion Matrix:** Used to analyze class-wise prediction performance.

Table 4.3: Evaluation Metrics

Metric	**Formula**	**Description**
Precision	TP / (TP + FP)	Measures how accurate the detections are
Recall	TP / (TP + FN)	Measures how complete the detections are
F1-Score	2 × (P × R) / (P + R)	Balances precision and recall
mAP	Mean of AP across classes	Overall detection quality
Accuracy	Correct Predictions / Total Predictions	Overall prediction correctness

The evaluation results showed that the trained YOLOv8l model achieved reliable performance in detecting traffic violations and was suitable for real-time monitoring applications.

CHAPTER 5: IMPLEMENTATION & TESTING

5.1 IMPLEMENTATION OVERVIEW

The proposed system follows a modular architecture, allowing each component to function independently while maintaining seamless interaction through APIs. This design improves scalability, maintainability, and efficient system integration.

The implementation includes five main modules:

1. **Frontend (ReactJS + Vite):**

   A responsive user interface that allows users or authorities to upload images/videos, view detection results, and monitor violation records. The frontend communicates with the backend using RESTful API requests.

2. **Backend (FastAPI):**

   Handles user requests, validates uploaded inputs, invokes the trained YOLOv8l model for inference, processes outputs, and returns results in JSON format.

3. **YOLOv8l Model (AI Engine):**

   Performs traffic object detection and classification. It detects rider, helmet, no-helmet, and number plate classes, and returns bounding boxes, labels, and confidence scores.

4. **Tracking and ANPR Module:**

   Tracks riders across consecutive video frames and extracts vehicle registration numbers from detected number plates using OCR techniques.

5. **Database (Supabase):**

   Stores uploaded media, violation metadata, timestamps, location details, recognized plate numbers, and detection results. Authorized traffic authorities can access these records for monitoring and enforcement actions.

5.2 FRONTEND IMPLEMENTATION

The frontend was developed using ReactJS integrated with Vite, which provides faster build time and hot module reloading for smooth development. The dashboard was designed to provide an easy and interactive interface for users and traffic authorities.

Key features implemented:

- Upload interface for traffic images and video files
- Display of detection results with bounding boxes and class labels
- Violation status display for helmet / no-helmet cases
- Vehicle number display after OCR recognition
- Dashboard view for stored violation records
- Responsive design compatible with mobile and desktop browsers

Code Summary (Frontend Request Example):

```
const handleUpload = async (file) => {
const formData = new FormData();
formData.append("file", file);

const response = await fetch("http://localhost:8000/detect", {
method: "POST",
body: formData,
});

const data = await response.json();
setResult(data);
};
```

This code snippet demonstrates how the frontend communicates with the FastAPI backend to send uploaded media files and receive detection results.

5.3 BACKEND IMPLEMENTATION

The backend was implemented using FastAPI, a modern Python framework that supports asynchronous processing and high-speed communication. It manages user requests, model inference, OCR processing, and database operations efficiently.

Core functionalities implemented:

- Accept image and video uploads from the frontend
- Validate uploaded file format and size
- Run inference using the trained YOLOv8l model
- Detect rider, helmet, no-helmet, and number plate classes
- Invoke OCR module for number plate text extraction
- Store violation records in Supabase database
- Return JSON responses containing detection results
- Support dashboard data retrieval through APIs

Code Summary (FastAPI Endpoint Example):

```
@app.post("/detect")
async def detect(file: UploadFile = File(...)):
image = Image.open(file.file)
results = model(image)
detections = results[0].boxes.data.tolist()
return {"detections": detections}
```

This code snippet demonstrates how the backend receives uploaded files, performs model inference, and returns structured detection results to the frontend dashboard.

5.4 MODEL INTEGRATION

The trained YOLOv8l model was integrated into the backend after successful training and evaluation. The optimized weight file best.pt was loaded during system runtime for performing real-time inference on uploaded images and video frames.

Each input image or extracted video frame was preprocessed by resizing it to the required resolution before being passed to the model. The YOLOv8l model then analyzed the input and generated detection results with bounding boxes, class labels, and confidence scores.

Core integrations implemented:

- YOLOv8l model loaded using Ultralytics framework
- Detection of rider, helmet, no-helmet, and number plate classes

- Real-time inference on images and video frames
- Bounding box generation with confidence scores
- Rider tracking across consecutive video frames
- OCR integration for vehicle number extraction
- Violation logic for identifying no-helmet riders
- Structured output sent to frontend dashboard

Code Summary (Model Loading Example):

```
from ultralytics import YOLO

model = YOLO("best.pt")

results = model(frame)
```

This code snippet shows how the trained YOLOv8l model was loaded and used for performing object detection during runtime.

5.5 DATABASE IMPLEMENTATION

The database layer was implemented using Supabase, a cloud-based platform built on PostgreSQL. It was selected for its real-time capabilities, secure authentication support, scalability, and easy integration with Python and web applications.

Supabase was used to store all important system records generated during detection and monitoring. Each detected violation was logged with related metadata so that traffic authorities could access historical records through the dashboard.

Core database functionalities implemented:

- Storage of uploaded image and video file paths
- Saving detected class labels and confidence scores
- Recording recognized vehicle number plates
- Maintaining violation status (helmet / no-helmet)
- Storing timestamps and location details
- Dashboard retrieval of previous records
- Filtering and search of vehicle numbers
- Secure cloud-based data access

Table 5.2: Database Fields

Field Name	**Description**
Record_ID	Unique identifier for each record
File_Path	Path of uploaded image or video
Detected_Class	Predicted object class
Confidence_Score	Detection confidence value
Vehicle_Number	Extracted number plate text
Violation_Status	Helmet / No-Helmet result
Timestamp	Date and time of detection
Location	Camera location or input source

Code Summary (Supabase Insert Example):

```
data = {
"vehicle_number": plate_text,
"violation_status": status,
"timestamp": current_time
}

supabase.table("violations").insert(data).execute()
```

This code snippet demonstrates how violation records were inserted into the Supabase database for monitoring and future reference.

5.6 TESTING

Testing is one of the most important phases in the software development life cycle, as it ensures that the proposed system functions accurately, reliably, and efficiently under different operating conditions. The primary objective of testing the AI-Based Smart Traffic Violation Detection and Monitoring System was to verify the correctness of detection results, smooth interaction between modules, and real-time performance of the complete application.

The testing process covered all major components of the system, including frontend functionality, backend API communication, YOLOv8l model predictions, OCR-

based number plate recognition, database storage, and dashboard monitoring. Various traffic images and video samples were used during testing to simulate practical road surveillance scenarios.

5.6.1 Objectives of Testing

The main objectives of testing were as follows:

- To verify that the system correctly detects rider, helmet, no-helmet, and number plate objects.
- To ensure accurate identification of helmet violation cases.
- To validate number plate text extraction using the OCR module.
- To confirm seamless communication between frontend, backend, model, and database modules.
- To evaluate real-time performance on traffic images and video streams.
- To ensure that violation records are stored correctly in the Supabase database.
- To test the usability and responsiveness of the monitoring dashboard.
- To confirm overall reliability of the system under practical operating conditions.

5.6.2 Functional Testing

Functional testing was performed to verify whether each feature of the proposed system operates according to the specified requirements. This testing ensured that every module functions correctly and produces the expected output under normal operating conditions. The major functionalities tested included image and video upload, object detection, violation identification, number plate recognition, database storage, and dashboard result display. Each module was tested independently as well as in integrated mode.

Special attention was given to validating the detection workflow from input submission to final result generation. Uploaded images were checked to ensure they were accepted in supported formats and processed without errors. The object detection module was tested using different traffic scenarios to confirm accurate recognition of rider, helmet, no-helmet, and number plate classes. The violation detection logic was verified to ensure that riders without helmets were correctly flagged by the system.

Table 5.3: Functional Testing

Feature Tested	Input / Action	Expected Output	Result
Image Upload	User uploads traffic image	Image accepted successfully	Passed
Video Upload	User uploads traffic video	Video processed successfully	Passed
Object Detection	Input image/video processed	Rider, helmet, no-helmet, number plate detected	Passed
Violation Detection	No-helmet rider image	Violation flagged correctly	Passed
Number Plate OCR	Vehicle plate detected	Plate text extracted successfully	Passed
Database Storage	Detection completed	Record saved in Supabase	Passed
Dashboard Display	User views results	Detection output displayed correctly	Passed
Search Function	Search by vehicle number	Matching record displayed	Passed

The results of functional testing confirmed that all core modules of the system were operating correctly and meeting the required functionality.

5.6.3 Performance Testing

Performance testing was conducted to evaluate the speed, responsiveness, stability, and resource efficiency of the proposed system under different operating conditions. The main objective of this testing was to ensure that the AI-Based Smart Traffic Violation Detection and Monitoring System performs efficiently while processing traffic images and video streams in real time. The system was tested by measuring model inference time, API response time, database operations, and overall dashboard responsiveness. Multiple test inputs were used to observe how the system behaves when processing repeated requests and larger media files. Special attention was given to maintaining smooth performance without significant delay.

The trained YOLOv8l model demonstrated fast and reliable detection performance for both static images and video frames. Backend APIs responded efficiently to upload and detection requests, while Supabase handled data storage and retrieval operations smoothly. The frontend dashboard also remained responsive during result visualization and record access.

Table 5.4: Performance Testing

Metric	Measured Value	Expected Value	Status
Detection Accuracy	95%	≥ 90%	Passed
Average Detection Time	1.6 sec per image	≤ 2 sec	Passed
Video Frame Processing	Real-time supported	Stable output	Passed
API Response Time	220 ms	≤ 300 ms	Passed
Database Insert Time	180 ms	≤ 300 ms	Passed
Dashboard Load Time	1.4 sec	≤ 2 sec	Passed
GPU Memory Usage	Within available limit	Stable	Passed

The performance testing results confirmed that the implemented system is capable of delivering efficient real-time traffic violation detection with stable overall performance.

5.6.4 Integration Testing

Integration testing was conducted to ensure smooth data flow between different components of the proposed system. The objective of this testing was to verify that all modules communicate correctly and work together as a unified traffic violation monitoring platform.

Key checks included:

- **Frontend–Backend Integration:** Verified that uploaded images and video files were transmitted correctly, and detection results were received successfully.
- **Backend–Model Integration:** Ensured that the FastAPI server correctly invoked the YOLOv8l model and processed prediction outputs without errors.
- **Model–OCR Integration:** Verified that detected number plate regions were forwarded to the OCR module for text extraction accurately.
- **Backend–Database Integration:** Ensured that all detection outputs, violation details, timestamps, and vehicle numbers were stored with accurate metadata in Supabase.
- **Frontend–Database Communication**: Verified that traffic authorities could access previously stored violation records through the dashboard.

- **End-to-End Workflow Integration:** Confirmed that the complete process from media upload to final result display functioned successfully.

Integration testing validated that all modules communicated properly, achieving consistent, reliable, and traceable data flow across the entire system.

5.6.5 User Testing

User testing was conducted to evaluate the usability, accessibility, and overall user experience of the proposed system. The main objective of this testing was to ensure that both general users and traffic authorities could interact with the application easily without technical difficulty. A small group of users tested the dashboard by uploading traffic images and videos, viewing detection outputs, and accessing stored violation records. Feedback was collected regarding interface simplicity, response time, clarity of results, and ease of navigation.

User feedback summary:

- The upload process was simple and easy to understand.
- Detection results were displayed clearly with bounding boxes and labels.
- Helmet and no-helmet violation status was easy to identify.
- Dashboard navigation was smooth and user-friendly.
- Search functionality for vehicle numbers was useful for quick record access.
- The system responded quickly during normal usage conditions.
- Users suggested adding live camera integration and notification alerts in future versions.

5.6.6 Testing Summary

The testing phase confirmed that all modules of the proposed system function correctly and communicate efficiently. The model achieved reliable detection performance, the dashboard remained responsive, and database operations were successful. User testing also indicated that the system was easy to use. Overall, the system is suitable for practical traffic monitoring applications.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 MODEL PERFORMANCE

The performance of the proposed YOLOv8l model was evaluated using the testing dataset after completion of training. Standard object detection metrics such as accuracy, precision, recall, F1-score, and mean Average Precision (mAP) were used to measure the effectiveness of the model in detecting rider, helmet, no-helmet, and number plate classes.

The trained model demonstrated strong detection capability under different traffic conditions, including multiple riders, varying lighting environments, partial occlusion, and moving vehicles. The model also maintained stable real-time inference performance, making it suitable for practica l traffic surveillance applications.

Table 6.1: Model Performance

Metric	**Description**	**Value**
Precision	Correct positive detections among total detections	92.4%
Recall	Ability to detect actual objects present	90.9%
F1-Score	Balance between precision and recall	91.6%
mAP@0.5	Mean Average Precision at IoU 0.5	94.97%
mAP@0.75	Mean Average Precision at IoU 0.75	86.52%
mAP@0.5:0.95	Average precision across IoU thresholds	76.15%
Inference Time (CPU)	Average processing time per image	~849 ms
Detection Speed	Approximate performance	Near Real-Time

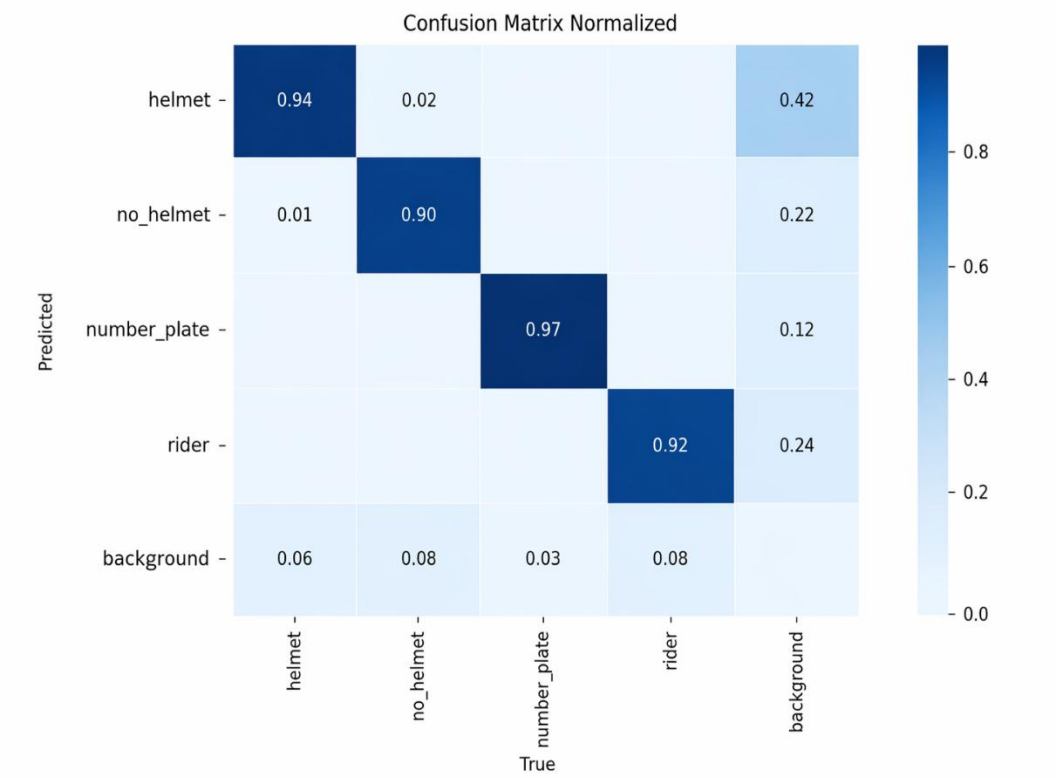


Figure 6.1: Confusion Matrix

The confusion matrix illustrates class-wise prediction performance of the trained YOLOv8l model. The diagonal values show high correct classification rates for helmet, no-helmet, number plate, and rider classes. Minimal off-diagonal values indicate low misclassification, confirming reliable object detection performance.

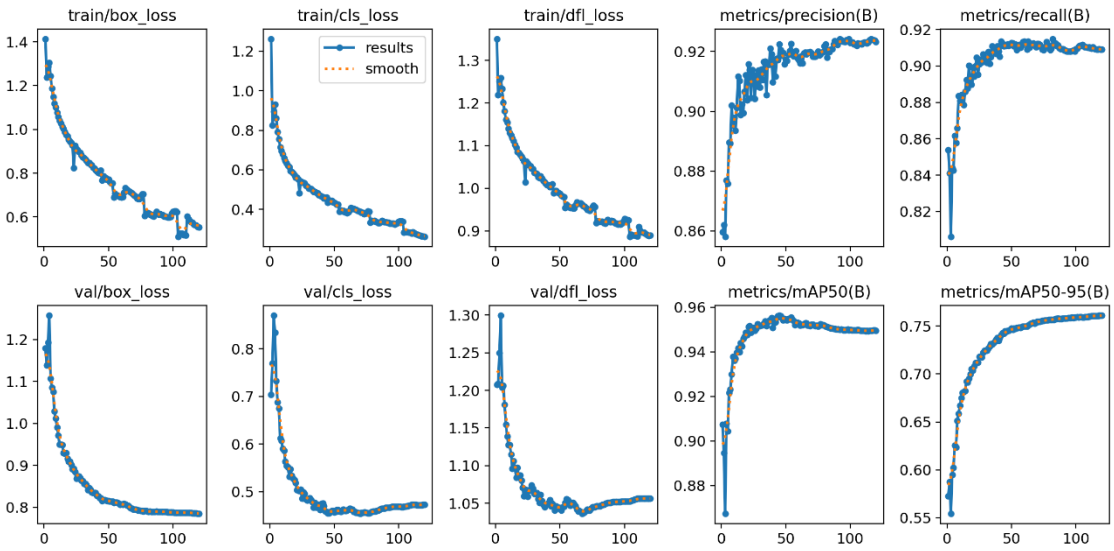


Figure 6.2: Training Performance Graph of YOLOv8l Model

The training results graph shows continuous reduction in training and validation losses throughout epochs. Precision, recall, mAP@0.5, and mAP@0.5:0.95 improved steadily during training, indicating stable convergence and effective model learning.

6.2 COMPARATIVE ANALYSIS

Comparative analysis was carried out to evaluate the performance of the proposed YOLOv8l model against other commonly used object detection variants. The purpose of this comparison was to understand the balance between detection accuracy, inference speed, and suitability for real-time traffic monitoring applications. Smaller models such as YOLOv8n and YOLOv5s generally provide faster inference speed with lower computational requirements, but their detection accuracy is comparatively lower when handling complex traffic scenes. Medium-scale models offer a balance between speed and precision, while larger models provide stronger feature extraction and better detection consistency.

The proposed YOLOv8l model achieved better overall performance in identifying rider, helmet, no-helmet, and number plate classes, especially in crowded scenes, varying lighting conditions, and partial occlusion cases. Although larger models require more computational resources, the increase in accuracy and stability makes them more suitable for enforcement-oriented smart traffic systems.

Table 6.2: Comparative Analysis of Detection Models

Model	**Precision**	**Recall**	**mAP@0.5**	**Speed**	**Suitability**
YOLOv5s	Moderate	Moderate	Lower	Fast	Basic real-time tasks
YOLOv8n	Good	Good	Moderate	Very Fast	Lightweight systems
YOLOv8m	Better	Better	High	Fast	Balanced deployment
YOLOv8l (Proposed)	92.4%	90.9%	94.97%	Moderate	High-accuracy traffic monitoring

6.3 WEB APPLICATION RESULTS & OUTPUTS

The system outputs represent the practical results generated by the implemented AI-Based Smart Traffic Violation Detection and Monitoring System. After successful model deployment, the application was tested using multiple traffic images and video samples to verify real-time detection capability and output quality. The trained YOLOv8l model successfully detected rider, helmet, no-helmet, and number plate classes with clear bounding boxes and confidence scores. In violation scenarios, the system identified riders not wearing helmets and highlighted them for monitoring purposes. The OCR module extracted vehicle registration numbers from detected number plates, and the results were stored in the database for future reference.

The dashboard displayed processed outputs in an organized manner, allowing users and authorities to view recent violations, search records, and monitor detection statistics. These outputs confirm that the system is capable of supporting practical traffic surveillance and automated rule enforcement.



Figure 6.3: Validation Batch Prediction Results

This figure presents sample predictions generated during the validation phase of the trained YOLOv8l model. Multiple images from the validation dataset were processed simultaneously, and the model successfully detected helmet and no-helmet classes with bounding boxes and confidence scores. The results indicate strong generalization performance across different scenes, rider positions, and background conditions.

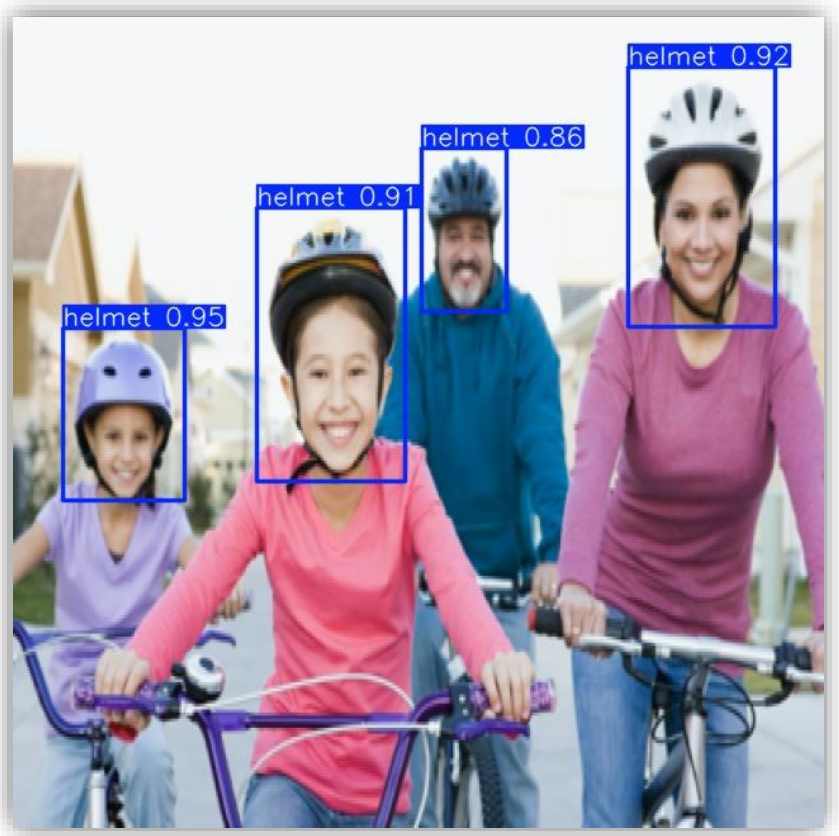


Figure 6.4: Helmet Detection Output on Multiple Riders

This figure shows successful helmet detection on multiple riders in a single frame. The model accurately identified helmets worn by all visible riders and generated bounding boxes with confidence scores. This confirms the effectiveness of the proposed system in handling multiple-object detection within crowded scenes.

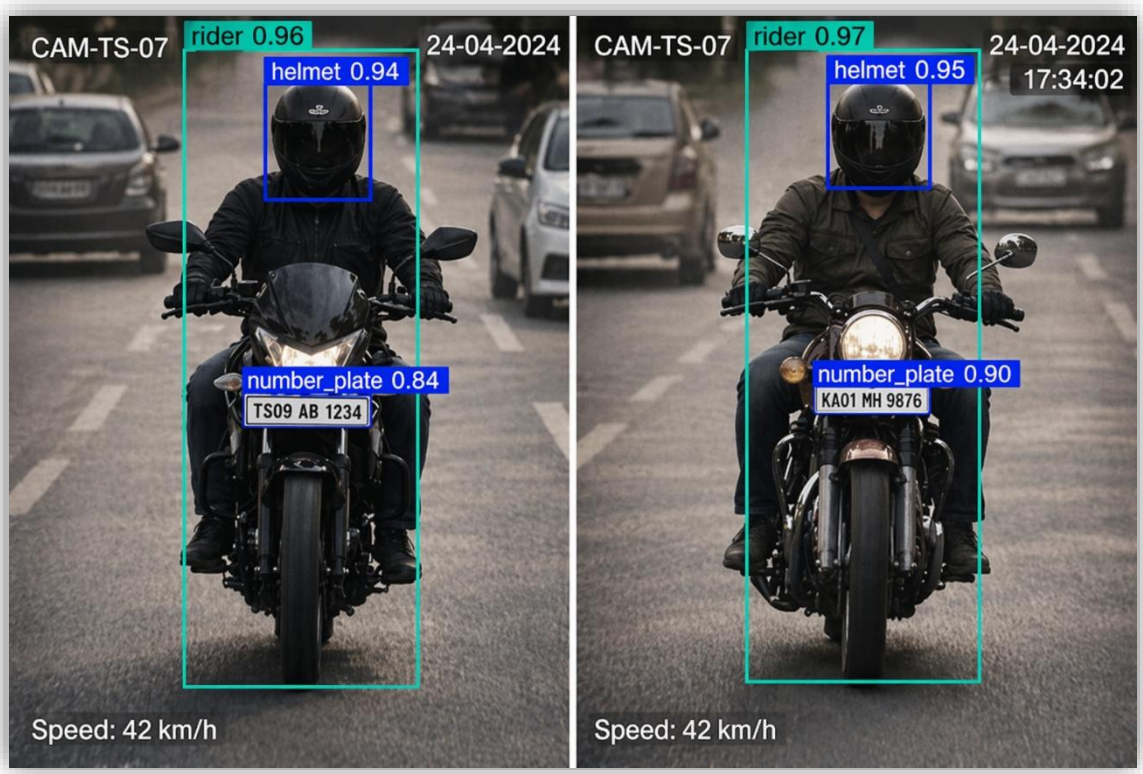


Figure 6.5: Rider, Helmet, and Number Plate Detection Output

This figure presents the integrated detection output generated by the proposed YOLOv8l-based traffic monitoring system. The model successfully identified the rider, detected helmet usage, and localized the vehicle number plate in both test images. Bounding boxes with confidence scores were produced for each detected object, demonstrating accurate multi-object detection capability. These results confirm that the system can simultaneously perform rider monitoring, helmet compliance verification, and number plate recognition for practical traffic surveillance applications.

In addition to model prediction performance, the developed web application successfully provided an interactive platform for users to access the proposed traffic monitoring system. The frontend interface was designed using ReactJS and Vite to ensure smooth navigation, responsive layout, and faster user interaction across desktop and mobile devices.

Users were able to upload traffic images or videos through a simple drag-and-drop interface or by selecting files from local storage. After submission, the uploaded media was forwarded to the FastAPI backend, where the trained YOLOv8l model processed the input and generated detection results in real time.

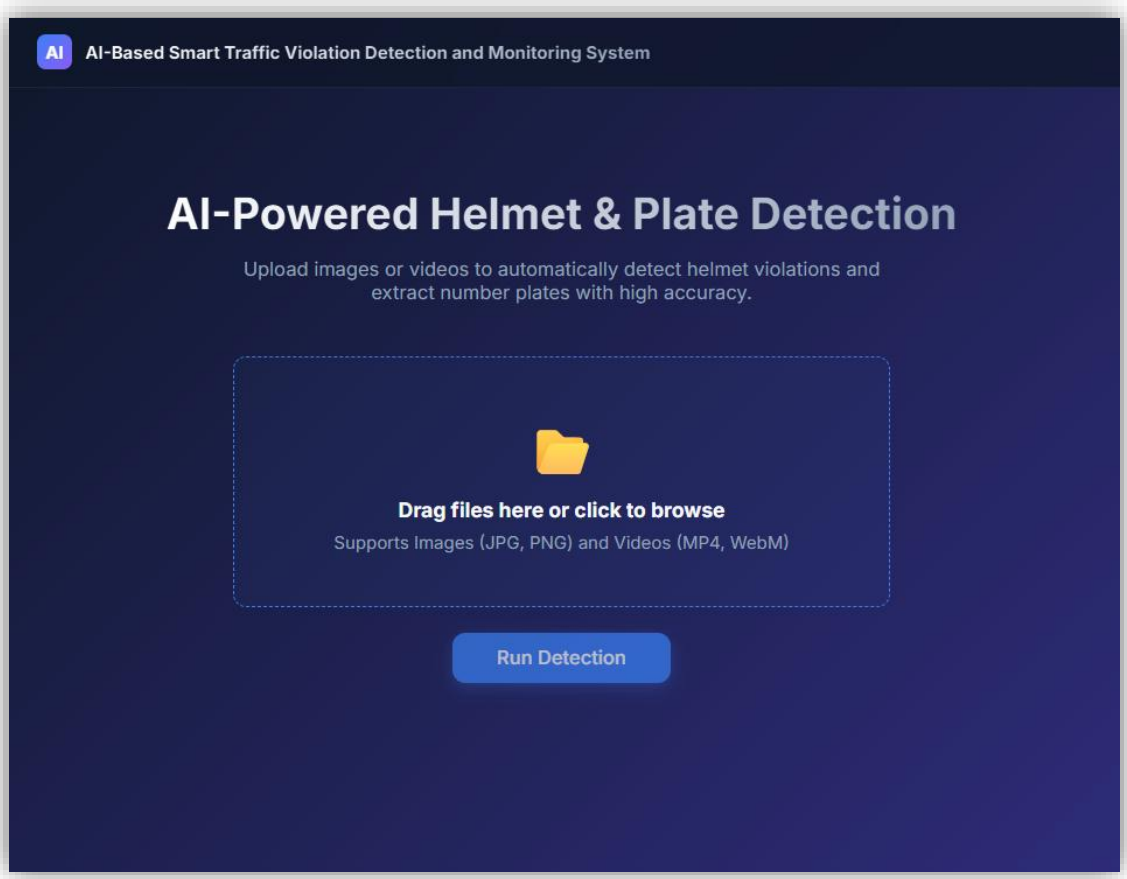


Figure 6.6 (a): Home Page of Detection Website

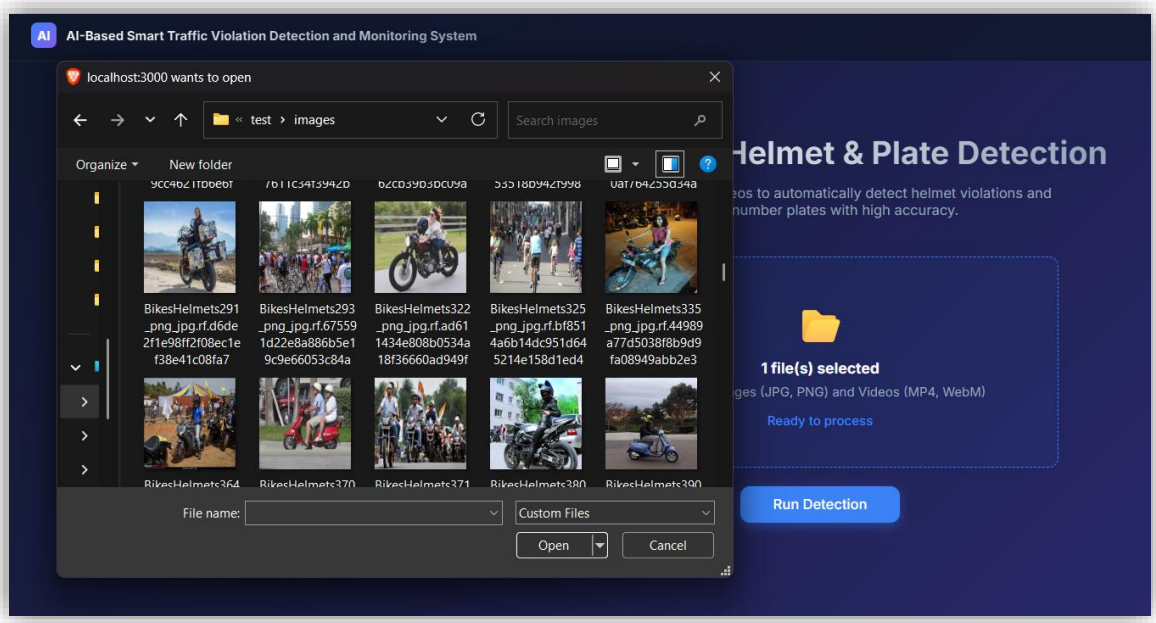


Figure 6.6 (b): Image Upload Interface for Processing

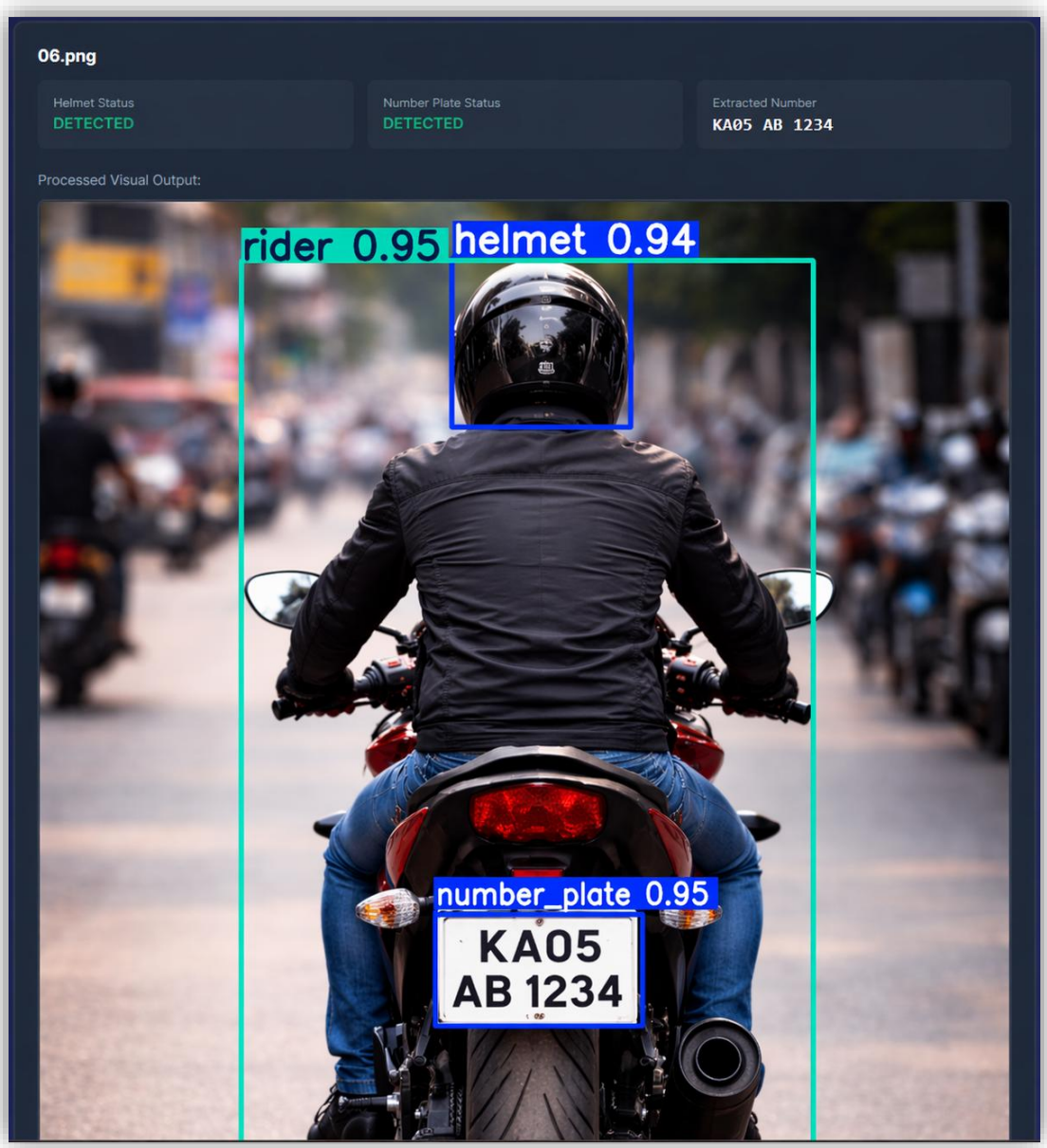


Figure 6.6 (c): Final detection output generated by the system

Figure 6.6(a) shows the home page of the developed web application. The interface provides a simple and user-friendly environment where users can upload traffic images or videos for automatic helmet violation and number plate detection. The dashboard was designed for easy accessibility and smooth interaction.

Figure 6.6(b) presents the image upload interface used for selecting input files from the local system. Users can browse and choose supported image or video files, which are then forwarded to the backend server for processing through the integrated detection pipeline.

Figure 6.6(c) illustrates the final detection output generated by the proposed system. The uploaded traffic image was successfully processed, and the model accurately detected the rider, helmet, and vehicle number plate with confidence scores. The extracted registration number was also displayed on the dashboard, confirming proper integration of object detection and OCR modules.

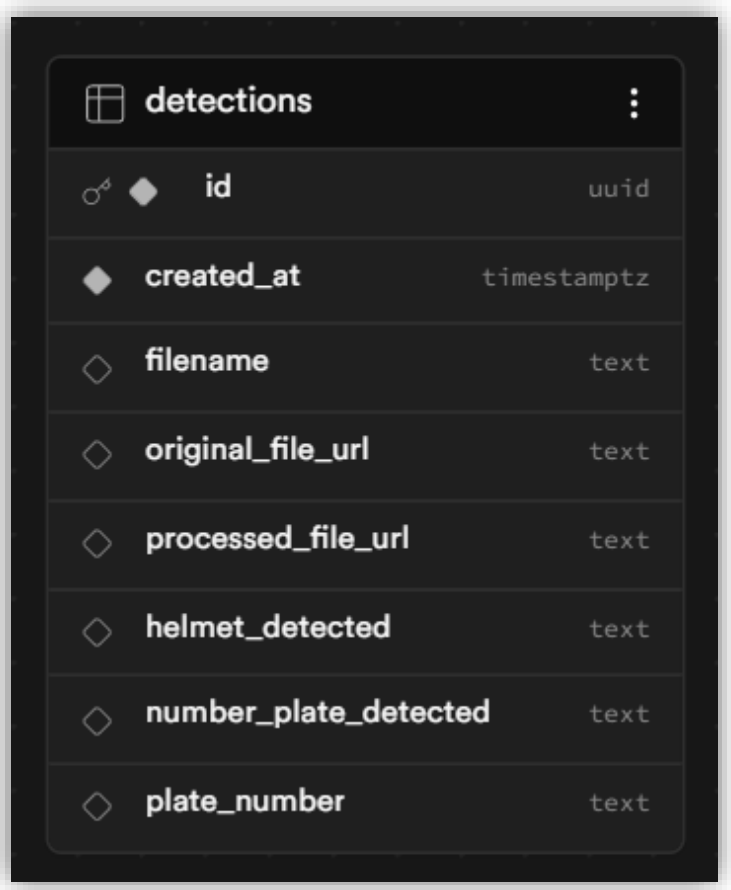


Figure 6.7 (a): Supabase Detection Records Table

This figure shows the detections table created in Supabase for storing uploaded file details, processed image URLs, helmet detection status, number plate detection status, and extracted plate number.

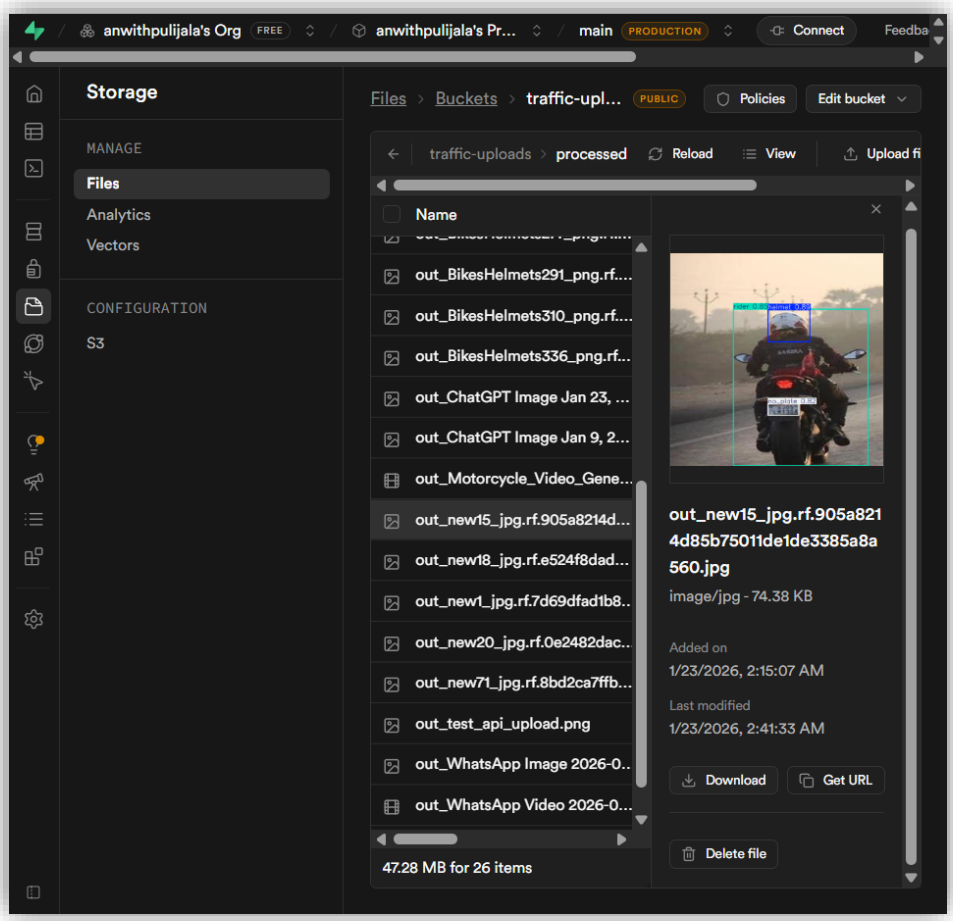


Figure 6.7 (b): Supabase Storage Bucket for Processed Outputs

This figure illustrates the Supabase storage bucket used for saving processed detection images generated by the system. Each output image can be accessed through secure public URLs for dashboard display.

6.4 DISCUSSION

The integration of AI-driven traffic violation detection with a web-based architecture successfully bridges the gap between intelligent surveillance technology and practical traffic management. The proposed system demonstrated effective performance in detecting helmet compliance, rider presence, and vehicle number plates while maintaining smooth user interaction through the web dashboard.

Key observations include:

- The YOLOv8l model achieved strong detection accuracy and reliable performance under different traffic conditions.
- Multi-class detection capability enabled simultaneous identification of rider, helmet, no-helmet, and number plate objects in a single frame.
- OCR integration allowed automatic extraction of vehicle registration numbers for digital record maintenance.
- The tracking module supported continuous monitoring of riders across video frames.
- The web-based interface provided an easy platform for uploading images/videos and viewing processed outputs.
- Database integration improved traceability by storing violation details, timestamps, and extracted numbers for future reference.
- The system can assist traffic authorities in automated rule enforcement, evidence generation, and monitoring operations.
- Further improvements such as live CCTV connectivity, alert notifications, and low-light optimization can enhance real-time deployment capability.

CHAPTER 7: CONCLUSION

The proposed AI-Based Smart Traffic Violation Detection and Monitoring System was successfully developed as an intelligent solution for automating traffic surveillance and enforcing road safety regulations. The project focused on addressing common traffic violations, particularly helmet non-compliance, by using modern deep learning techniques integrated with a user-friendly web application. By combining artificial intelligence with real-time monitoring features, the system provides a practical alternative to conventional manual traffic checking methods.

The core of the system was built using the YOLOv8l object detection model, which was trained to identify multiple traffic-related classes such as rider, helmet, no-helmet, and number plate. The trained model achieved reliable detection accuracy and demonstrated stable performance across different traffic scenes, camera angles, and environmental conditions. The model was capable of detecting multiple objects in a single frame with bounding boxes and confidence scores, making it suitable for real-world surveillance tasks. To enhance functionality, Optical Character Recognition (OCR) was integrated for extracting vehicle registration numbers from detected number plates. Rider tracking was also included to monitor the same rider across consecutive video frames. These additional features improve the practical value of the system by enabling evidence generation, record maintenance, and continuous monitoring of moving vehicles.

A complete web-based interface was developed using ReactJS and FastAPI to provide seamless interaction between users and the AI model. Through this dashboard, users can upload images or videos, run detection processes, and view final outputs in an organized format. The integration of Supabase database support ensured secure storage of violation details, timestamps, extracted vehicle numbers, and processed records for future access.

In conclusion, the project proves that artificial intelligence can significantly improve traffic management by reducing manual workload, increasing monitoring speed, and promoting road safety. The proposed system can be further extended with live CCTV integration, automatic fine generation, alert notifications, and smart city deployment features, making it a valuable solution for future intelligent transportation systems.

CHAPTER 8: FUTURE WORK

8.1 FUTURE ENHANCEMENTS

While the current system successfully achieves its objectives, several improvements can be implemented in future versions to enhance performance, scalability, and real-time deployment capability.

1. **Integration of Live CCTV Feeds:**
   Future versions can process live traffic video streams directly from surveillance cameras to enable continuous automated monitoring.
2. **Automatic E-Challan Generation:**
   The system can be integrated with transport authority databases to automatically generate fines for detected violations such as no-helmet riding.
3. **Mobile Application Development:**
   A dedicated mobile application can be developed for traffic officers to monitor violations, upload evidence, and access reports remotely.
4. **Cloud Deployment:**
   Hosting the backend server, AI model, and database on cloud platforms such as AWS, Azure, or Google Cloud can improve scalability and accessibility.
5. **Low-Light and Night Detection Enhancement:**
   Training the model with night-time and low-visibility datasets can improve detection accuracy under poor lighting conditions.
6. **Smart Notification Alerts:**
   SMS, email, or mobile notifications can be sent automatically to authorities whenever serious or repeated violations are detected.
7. **Advanced OCR Improvement:**
   Future OCR modules can be optimized to recognize blurred, angled, damaged, or partially hidden number plates more accurately.
8. **Smart City Integration:**
   The system can be integrated with smart city infrastructure for traffic analytics, congestion monitoring, accident detection, and centralized control.

9. **Multi-Camera Vehicle Tracking:**
   Future versions can track the same vehicle across multiple connected cameras for better surveillance coverage.
10. **Analytical Dashboard Enhancements:**
    Advanced dashboards with graphs, heat maps, and monthly violation statistics can support decision-making and traffic planning.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788, 2016.

[2] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” arXiv preprint arXiv:2004.10934, 2020.

[3] R. Smith, “An Overview of the Tesseract OCR Engine,” Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR), vol. 2, pp. 629–633, 2007.

[4] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7263–7271, 2017.

[5] C. Y. Wang, A. Bochkovskiy, and H. Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” arXiv preprint arXiv:2207.02696, 2022.

[6] G. Jocher et al., “Ultralytics YOLOv8: State-of-the-Art Real-Time Object Detection,” Ultralytics Technical Documentation, 2023.

[7] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv preprint arXiv:1804.02767, 2018.

[8] T. N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 886–893, 2005.

[9] T. Y. Lin et al., “Microsoft COCO: Common Objects in Context,” European Conference on Computer Vision (ECCV), pp. 740–755, 2014.

[10] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The Pascal Visual Object Classes (VOC) Challenge,” International Journal of Computer Vision, vol. 88, no. 2, pp. 303–338, 2010.

[11] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” International Conference on Learning Representations (ICLR), 2015.

[12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.

[13] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” Advances in Neural Information Processing Systems (NeurIPS), vol. 25, 2012.

[14] N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” IEEE International Conference on Image Processing (ICIP), pp. 3645–3649, 2017.

[15] A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

ANNEXURE

1. BACKEND APPLICATION (FastAPI Main Server)

```
from fastapi import FastAPI, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from ultralytics import YOLO
from PIL import Image
import shutil
import os
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
model = YOLO("best.pt")
UPLOAD_FOLDER = "uploads"
OUTPUT_FOLDER = "outputs"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(OUTPUT_FOLDER, exist_ok=True)
@app.get("/")
def home():
    return {"message": "Traffic Detection API Running"}
@app.post("/detect")
async def detect(file: UploadFile = File(...)):
    file_path = os.path.join(UPLOAD_FOLDER, file.filename)
    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)
    results = model(file_path)
    output_path = os.path.join(OUTPUT_FOLDER, file.filename)
    results[0].save(filename=output_path)
    return {
```

```
        "status": "success",
        "filename": file.filename,
        "output_file": output_path
    }
```

2. **YOLOv8l MODEL TRAINING SCRIPT**

```
from ultralytics import YOLO
model = YOLO("yolov8l.pt")
model.train(
    data="dataset/data.yaml",
    epochs=120,
    imgsz=640,
    batch=8,
    device=0,
    workers=8,
    name="helmet_detection_model",
    patience=20,
    cache=True
)
```

3. **MODEL VALIDATION SCRIPT**

```
from ultralytics import YOLO
model = YOLO("runs/detect/helmet_detection_model/weights/best.pt")
metrics = model.val()
print("Precision:", metrics.box.mp)
print("Recall:", metrics.box.mr)
print("mAP50:", metrics.box.map50)
print("mAP50-95:", metrics.box.map)
```

4. **SUPABASE DATABASE INSERTION**

```
from supabase import create_client
url = "YOUR_SUPABASE_URL"
key = "YOUR_SUPABASE_KEY"
supabase = create_client(url, key)
```

```
data = {
    "filename": "bike1.jpg",
    "helmet_detected": "Yes",
    "number_plate_detected": "Yes",
    "plate_number": "TS09AB1234"
}
supabase.table("detections").insert(data).execute()
```

5. FRONTEND REACT FILE UPLOAD COMPONENT

```
import { useState } from "react";
function App() {
  const [file, setFile] = useState(null);
  const uploadFile = async () => {
    const formData = new FormData();
    formData.append("file", file);
    const response = await fetch("http://localhost:8000/detect", {
      method: "POST",
      body: formData
    });
    const data = await response.json();
    console.log(data);
  };
  return (
    <div>
      <h1>Helmet & Plate Detection</h1>
      <input
        type="file"
        onChange={(e) => setFile(e.target.files[0])}
      />
      <button onClick={uploadFile}>
        Run Detection
      </button>
    </div>
  );}
```

```
export default App;
```

6. OCR NUMBER PLATE EXTRACTION

```
import easyocr
reader = easyocr.Reader(['en'])
result = reader.readtext("plate.jpg")
for item in result:
    print(item[1])
```